
19 Data Structures for Disjoint Sets

Some applications involve grouping n distinct elements into a collection of disjoint sets—sets with no elements in common. These applications often need to perform two operations in particular: finding the unique set that contains a given element and uniting two sets. This chapter explores methods for maintaining a data structure that supports these operations.

Section 19.1 describes the operations supported by a disjoint-set data structure and presents a simple application. Section 19.2 looks at a simple linked-list implementation for disjoint sets. Section 19.3 presents a more efficient representation using rooted trees. The running time using the tree representation is theoretically superlinear, but for all practical purposes it is linear. Section 19.4 defines and discusses a very quickly growing function and its very slowly growing inverse, which appears in the running time of operations on the tree-based implementation, and then, by a complex amortized analysis, proves an upper bound on the running time that is just barely superlinear.

19.1 Disjoint-set operations

A *disjoint-set data structure* maintains a collection $\mathcal{S} = \{S_1, S_2, \dots, S_k\}$ of disjoint dynamic sets. To identify each set, choose a *representative*, which is some member of the set. In some applications, it doesn't matter which member is used as the representative; it matters only that if you ask for the representative of a dynamic set twice without modifying the

set between the requests, you get the same answer both times. Other applications may require a prespecified rule for choosing the representative, such as choosing the smallest member in the set (for a set whose elements can be ordered).

As in the other dynamic-set implementations we have studied, each element of a set is represented by an object. Letting x denote an object, we'll see how to support the following operations:

MAKE-SET(x), where x does not already belong to some other set, creates a new set whose only member (and thus representative) is x .

UNION(x, y) unites two disjoint, dynamic sets that contain x and y , say S_x and S_y , into a new set that is the union of these two sets. The representative of the resulting set is any member of $S_x \cup S_y$, although many implementations of **UNION** specifically choose the representative of either S_x or S_y as the new representative. Since the sets in the collection must at all times be disjoint, the **UNION** operation destroys sets S_x and S_y , removing them from the collection $\mathcal{S}$. In practice, implementations often absorb the elements of one of the sets into the other set.

FIND-SET(x) returns a pointer to the representative of the unique set containing x .

Throughout this chapter, we'll analyze the running times of disjoint-set data structures in terms of two parameters: n , the number of **MAKE-SET** operations, and m , the total number of **MAKE-SET**, **UNION**, and **FIND-SET** operations. Because the total number of operations m includes the n **MAKE-SET** operations, $m \geq n$. The first n operations are always **MAKE-SET** operations, so that after the first n operations, the collection consists of n singleton sets. Since the sets are disjoint at all times, each **UNION** operation reduces the number of sets by 1. After $n - 1$ **UNION** operations, therefore, only one set remains, and so at most $n - 1$ **UNION** operations can occur.

An application of disjoint-set data structures

One of the many applications of disjoint-set data structures arises in determining the connected components of an undirected graph (see Section B.4). Figure 19.1(a), for example, shows a graph with four connected components.

The procedure **CONNECTED-COMPONENTS** on the following page uses the disjoint-set operations to compute the connected components of a graph. Once the **CONNECTED-COMPONENTS** procedure has preprocessed the graph, the procedure **SAME-COMPONENT** answers queries about whether two vertices belong to the same connected component. In pseudocode, we denote the set of vertices of a graph G by $G.V$ and the set of edges by $G.E$.

The procedure **CONNECTED-COMPONENTS** initially places each vertex v in its own set. Then, for each edge (u, v) , it unites the sets containing u and v . By Exercise 19.1-2, after all the edges are processed, two vertices belong to the same connected component if and only if the objects corresponding to the vertices belong to the same set. Thus **CONNECTED-COMPONENTS** computes sets in such a way that the procedure **SAME-COMPONENT** can determine whether two vertices are in the same connected component. Figure 19.1(b) illustrates how **CONNECTED-COMPONENTS** computes the disjoint sets.

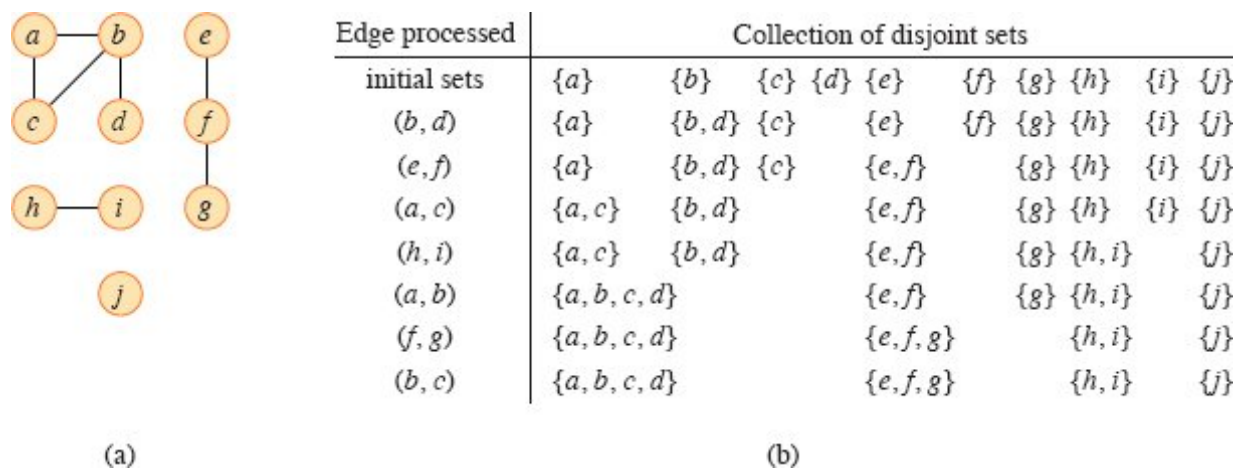


Figure 19.1 (a) A graph with four connected components: $\{a, b, c, d\}$, $\{e, f, g\}$, $\{h, i\}$, and $\{j\}$. (b) The collection of disjoint sets after processing each edge.

CONNECTED-COMPONENTS(G)

```

1 for each vertex  $v \in G.V$ 
2   MAKE-SET( $v$ )
3 for each edge  $(u, v) \in G.E$ 
4   if FIND-SET( $u$ )  $\neq$  FIND-SET( $v$ )
5     UNION( $u, v$ )

SAME-COMPONENT( $u, v$ )
1 if FIND-SET( $u$ ) == FIND-SET( $v$ )
2   return TRUE
3 else return FALSE

```

In an actual implementation of this connected-components algorithm, the representations of the graph and the disjoint-set data structure would need to reference each other. That is, an object representing a vertex would contain a pointer to the corresponding disjoint-set object, and vice versa. Since these programming details depend on the implementation language, we do not address them further here.

When the edges of the graph are static—not changing over time—depth-first search can compute the connected components faster (see Exercise 20.3-12 on page 572). Sometimes, however, the edges are added dynamically, with the connected components updated as each edge is added. In this case, the implementation given here can be more efficient than running a new depth-first search for each new edge.

Exercises

19.1-1

The CONNECTED-COMPONENTS procedure is run on the undirected graph $G = (V, E)$, where $V = \{a, b, c, d, e, f, g, h, i, j, k\}$, and the edges of E are processed in the order (d, i) , (f, k) , (g, i) , (b, g) , (a, h) , (i, j) , (d, k) , (b, j) , (d, f) , (g, j) , (a, e) . List the vertices in each connected component after each iteration of lines 3–5.

19.1-2

Show that after all edges are processed by CONNECTED-COMPONENTS, two vertices belong to the same connected component if and only if they belong to the same set.

19.1-3

During the execution of CONNECTED-COMPONENTS on an undirected graph $G = (V, E)$ with k connected components, how many times is FIND-SET called? How many times is UNION called? Express your answers in terms of $|V|$, $|E|$, and k .

19.2 Linked-list representation of disjoint sets

Figure 19.2(a) shows a simple way to implement a disjoint-set data structure: each set is represented by its own linked list. The object for each set has attributes *head*, pointing to the first object in the list, and *tail*, pointing to the last object. Each object in the list contains a set member, a pointer to the next object in the list, and a pointer back to the set object. Within each linked list, the objects may appear in any order. The representative is the set member in the first object in the list.

With this linked-list representation, both MAKE-SET and FIND-SET require only $O(1)$ time. To carry out MAKE-SET(x), create a new linked list whose only object is x . For FIND-SET(x), just follow the pointer from x back to its set object and then return the member in the object that *head* points to. For example, in Figure 19.2(a), the call FIND-SET(g) returns f .